Calibrating Self-Exciting Event Counts with a Learned Forward Operator

Technical report

Alberto Gennaro

24 June 2026

1 Scope

This report describes a method for fitting multivariate self-exciting (Hawkes) models to *interval-censored* event data — counts aggregated into fixed time bins — together with bin-level covariates, and the role a learned neural operator plays in that fit. Section 2 states the data and the model. Section 3 is the core of the report: it gives the calibration procedure step by step for a concrete, common case (covariates in the baseline only, constant excitation, weekly bins). Section 4 defines the forward operator and reports its accuracy. Sections 5–6 cover parameter recovery and the downstream ranking application. Section 8 lists limitations. All results are on synthetic data at CPU scale.

2 Problem setting and model

Data. There are M event streams ("sectors"), observed over W time bins (e.g. weeks) with endpoints $0 = a_0 < a_1 < \cdots < a_W = T$. For each bin i and sector m we observe an event count $C_{i,m} \in \mathbb{N}$. We also observe a covariate vector $X_i \in \mathbb{R}^p$ per bin (for example a popularity score, a sector indicator, a macro regime). Event times within a bin are *not* observed.

Model. Each sector m is driven by a conditional intensity

$$\lambda_m(t) = s_m(t) + \sum_{j=1}^{M} \sum_{t_k^{(j)} < t} A_{m,j}\, e^{-\theta\,(t - t_k^{(j)})}, \tag{1}$$

with a covariate-driven, log-linear baseline and a constant excitation matrix:

$$s_m(t) = \exp\big(\gamma_{m,0} + \gamma_m^\top X(t)\big), \qquad X(t) = X_i \text{ for } t \in (a_{i-1}, a_i], \tag{2}$$

and $A_{m,j} \geq 0$ the excitation of sector m by sector j, decaying at rate θ. In this report the excitation A does *not* depend on covariates; the covariates enter only through the baseline. (The covariate-modulated-excitation case is a direct extension, noted in Section 8.)

The quantity we can compute from counts. The Hawkes likelihood needs event times, which we do not have. We therefore work with the Mean Behavior Poisson Process (MBPP): the expected intensity $\xi = \mathbb{E}[\lambda]$ is deterministic and solves the Volterra equation

$$\xi(t) = s(t) + \int_0^t \Phi(t-u)\,\xi(u)\,\mathrm{d}u, \qquad \Phi_{m,j}(\tau) = A_{m,j}\,e^{-\theta\tau}. \tag{3}$$

By Watanabe's characterization the process with deterministic compensator $\Xi(t) = \int_0^t \xi$ is an inhomogeneous Poisson process, so the bin counts are independent Poisson variables,

$$C_{i,m} \sim \mathrm{Poisson}(\Xi_{i,m}), \qquad \Xi_{i,m} = \int_{a_{i-1}}^{a_i} \xi_m(t)\,\mathrm{d}t. \tag{4}$$

Calibration is maximum likelihood for this Poisson model. Up to a constant, the negative log-likelihood in the parameters $\Theta = (\gamma, A, \theta)$ is

$$\mathcal{L}(\Theta) = \sum_{i=1}^{W}\sum_{m=1}^{M} \Big(\Xi_{i,m}(\Theta) - C_{i,m}\,\log \Xi_{i,m}(\Theta)\Big). \tag{5}$$

3 Calibration procedure

Evaluating $\mathcal{L}$ in (5) requires the bin compensators $\Xi_{i,m}(\Theta)$, which come from solving (3) for the current parameters. That forward solve is the only expensive step, and it is repeated at every optimiser iteration. The method replaces it with a single call to a forward operator that has been trained, once, to solve (3) for any instance in the relevant family.

The forward operator. Let G_φ denote a neural operator (weights φ) that approximates the solution map

$$G_\varphi : \ (s(\cdot),\, A) \ \longmapsto \ \xi(\cdot), \tag{6}$$

i.e. it maps a baseline path s (sampled on a fixed time grid) and an excitation matrix A to the mean-intensity path ξ on that grid, in one forward pass. Section 4 describes how G_φ is built and trained; here we use it as a black-box, differentiable solver.

Offline (once). Train G_φ over the distribution of instances the fit will query: baseline paths of the form (2) spanning the plausible range of (γ, X), and subcritical excitation matrices A. Training minimises the residual of (3) (Section 4); the trained operator is then reused for every dataset.

Online (per dataset). Fit Θ by gradient descent on (5), using G_φ as the forward map. One iteration:

1. **Baseline path from covariates.** With the current γ, evaluate the piecewise-constant baseline on the grid, $s_m(t_k) = \exp(\gamma_{m,0} + \gamma_m^\top X(t_k))$, using the observed weekly covariates X_i.
2. **Forward solve.** $\xi = G_\varphi(s, A)$ — a single network pass, no integral-equation solve.
3. **Bin the compensator.** $\Xi_{i,m} = \int_{a_{i-1}}^{a_i} \xi_m$, computed by the trapezoid rule on the grid.
4. **Likelihood.** Evaluate $\mathcal{L}(\Theta)$ from (5) against the observed counts $C_{i,m}$.

5. **Gradient and step.** Because G_φ is differentiable and s depends on γ in closed form, obtain $\nabla_\Theta \mathcal{L}$ by automatic differentiation through the whole chain (no finite differences, no solver loop) and take a descent step.

The output is the fitted baseline coefficients γ (the covariate effects), the excitation matrix A, and the decay θ. Standard errors follow from the observed information; a Bayesian posterior is available by sampling (5) plus a prior.

What the operator does and does not change. The operator changes only how the forward solve in step 2 is computed; the statistical model (1)–(5) and the estimator are unchanged. For the baseline-only case of this section the forward solve also has an exact, inexpensive numerical solution (the kernel is a convolution), so the operator is used here for speed and for clean autodiff gradients; its decisive advantage appears in the covariate-modulated-excitation case (Section 8), where the equation is no longer a convolution and the per-iteration solve is the bottleneck.

4 The forward operator

Architecture. G_φ is a DeepONet: a branch network encodes the instance — the baseline path s sampled on the grid and the excitation matrix A — and a trunk network encodes the query time; their inner product gives $\xi(t)$. The operator must be conditioned on the instance it is solving: a network that does not receive A (or receives only the forcing while the system varies) is not a function on the family — the same input corresponds to many correct outputs — and cannot be fit below roughly 35–50% relative error. Conditioning on (s, A) removes this.

Training objective. On a grid $\{t_k\}$, write the discretised version of (3) as $\xi = s + A\,(G\xi)$, where G is the lower-triangular matrix implementing $\int_0^t e^{-\theta(t-u)}(\cdot)\,\mathrm{d}u$. The residual

$$R[\xi] = \xi - s - A\,(G\xi) \tag{7}$$

is linear in ξ, so it has a unique zero at the exact solution. The operator is trained to drive R to zero,

$$\min_{\varphi}\ \mathbb{E}_{(s,A)}\left[\frac{\|R[G_\varphi(s,A)]\|^2}{\|G_\varphi(s,A)\|^2}\right] \ +\ \text{(supervised term on a set of exact anchor solutions)}, \tag{8}$$

where the relative normalisation keeps large-intensity instances from dominating, and the anchor term (a small set of solutions from the exact solver) accelerates and stabilises training. No labelled solutions are required for the residual term.

Accuracy. Trained over instances of dimension M and evaluated against the exact multivariate solver on 400 held-out instances per dimension:

Trained on noise-corrupted targets and tested against the clean solver, the error remains within twice its noiseless value up to about 25% multiplicative noise.

5 Parameter recovery

Two routes recover the parameters from an observed intensity (or, through (4), from counts).

M (sectors)	instance dim $(M+M^2)$	rel-L_2 mean	median	p90	training time
3	12	0.55%	0.47%	0.87%	118 s
5	30	1.52%	1.36%	2.40%	161 s
8	72	2.03%	1.85%	3.11%	242 s

Table 1: Relative L_2 error of G_φ against the exact solver, held out. Error is approximately constant across the branching ratio and uniform across sectors. One evaluation takes about 3 ms. Source: `experiments/exp17_pino_jax.py`.

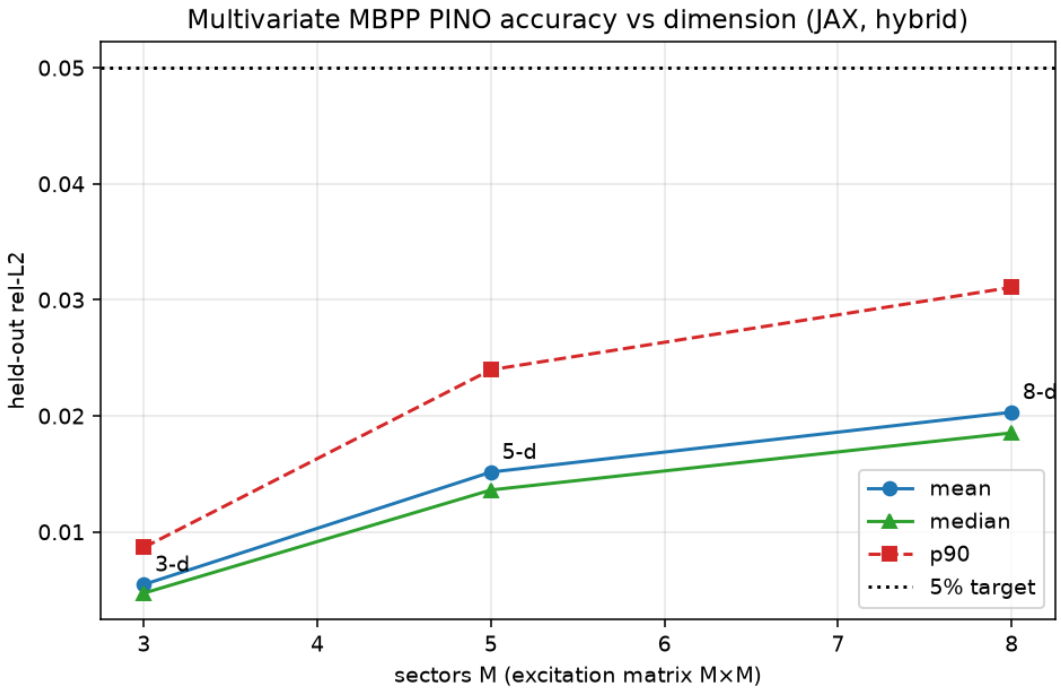

Figure 1: Forward operator versus the exact multivariate solver, by dimension and branching ratio.

The first is the likelihood fit of Section 3 itself, which returns (γ, A, θ) with standard errors. The second is direct inversion: minimise $\|G_\varphi(s, A) - \xi_{\text{obs}}\|^2 / \|\xi_{\text{obs}}\|^2$ over the parameters by gradient descent through the operator. On clean data both the covariate coefficients and the excitation matrix are recovered to within a few percent (covariate coefficients to $\leq 0.1\%$, excitation to $\leq 2.5\%$ at $M \leq 5$). Under observation noise the covariate coefficients remain the well-determined part (within 5% at noise level $\sigma = 0.02$ on a single path), while the excitation matrix is more sensitive and is recovered by averaging R repeated observations, with error scaling as $\sigma/\sqrt{R}$. A one-shot regression network that maps counts directly to parameters is fast but only weakly accurate (entry correlation about 0.38, overall stability correlation about 0.48) and is suitable as an initialisation, not as a final estimate. Source: `experiments/exp18_covariate_inverse.py`.

6 Application: ranking the next event

The downstream task is to rank the sectors (or firms) most likely to produce an event within a horizon h. Given a fitted model, the rank score is the probability of at least one event of stream m in $(t, t+h]$,

$$p_m(t,h) = 1 - \exp\Big(-\int_t^{t+h} \xi_m(u)\,\mathrm{d}u\Big), \tag{9}$$

the compensator increment, which the forward operator already produces.

An alternative model class for the same task is survival analysis with a ranking objective. The time to the next event is modelled by a discrete-time competing-risks hazard; the model is fitted by the survival likelihood (a masked Bernoulli over time bins) plus a pairwise ranking loss on the cumulative-incidence function, and the population baseline is the Kaplan–Meier curve. On a synthetic firm population the fitted model attains a concordance index of 0.687 against 0.617 for a single-covariate baseline (0.5 is random), with calibrated survival curves and a Recall@k improvement over the baseline (Figure 2). The relationship to the Hawkes model is that a cause-specific hazard is a per-stream intensity; survival lets it depend on covariates but not on the event history of other streams, which the Hawkes excitation adds. Detail: `SURVIVAL_RANKING.md`, `experiments/exp16_survival.py`.

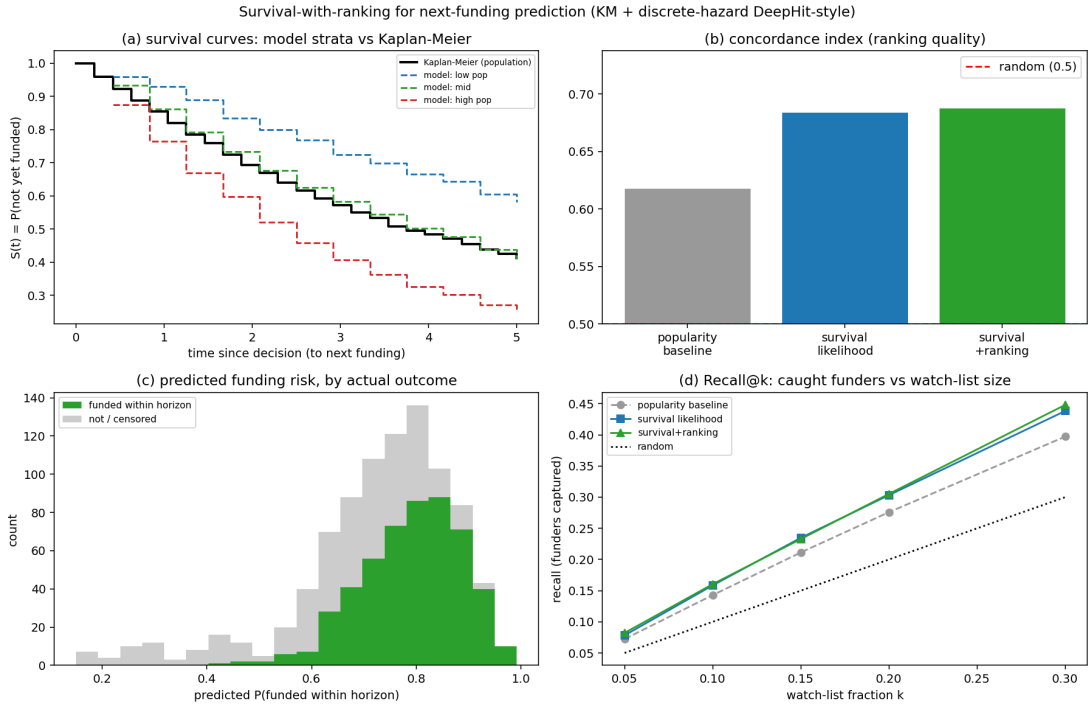

Figure 2: Survival model with a ranking objective: fitted survival curves against the Kaplan–Meier baseline (a); concordance index (b); predicted risk by outcome (c); Recall@k (d).

7 Software and reproducibility

The model, fitters, exact solvers, operator, and experiments are in the `hawkes_calibration` package. The counts-based fitters are `fit_mbpp_ic_covariates` (baseline covariates) and `fit_mbpp_ic_excitation_mul`

(covariate-modulated excitation, multivariate); the forward operator is in `hawkes_calibration/operators`; goodness-of-fit (time-rescaling) and a Bayesian posterior are provided. The automated test suite passes (53/53). Reproduction commands accompany each experiment script.

8 Limitations

- Results are on synthetic data at CPU scale (small networks, thousands of training instances). Validation at larger scale and on real binned-count series is required before drawing conclusions about absolute accuracy.
- Covariate-modulated excitation, $A_{m,j}(t) = A_{m,j} \exp(\delta^\top Z(t))$, uses the same procedure as Section 3 with the operator additionally conditioned on (δ, Z); this is the case where the per-iteration solve is most costly and the operator most useful. It is implemented in the exact solver and fitter; the operator for this case is the immediate next step.
- The branching ratio (overall stability) is well determined from counts; the decay θ and the individual cross-excitation entries are weakly identified, the more so the coarser the bins.
- For M beyond roughly 15, the M^2 excitation matrix should be encoded through a low-rank parameterisation rather than passed entry-wise to the operator.

9 References

Checked against source (authors, venue, year, link) on 24 June 2026.

Model and estimation

1. Rizoiu, Soen, Li, Calderon, Dong, Menon, Xie (2022). Interval-censored Hawkes processes. JMLR 23(1):1–84. https://www.jmlr.org/papers/v23/21-0917.html
2. Hawkes (1971). Spectra of some self-exciting and mutually exciting point processes. Biometrika 58(1):83–90.
3. Hawkes & Oakes (1974). A cluster process representation of a self-exciting process. J. Appl. Probab. 11(3).
4. Daley & Vere-Jones (2003). An Introduction to the Theory of Point Processes, Vol. I. Springer.
5. Banerjee, Merugu, Dhillon, Ghosh (2005). Clustering with Bregman divergences. JMLR 6.

Neural operators and point processes

6. Li, Kovachki, Azizzadenesheli, Liu, Bhattacharya, Stuart, Anandkumar (2021). Physics-Informed Neural Operator for Learning PDEs. https://arxiv.org/abs/2111.03794
7. Lu, Jin, Pang, Zhang, Karniadakis (2021). Learning nonlinear operators via DeepONet. Nature Machine Intelligence 3:218–229.
8. Du, Dai, Trivedi, Upadhyay, Gomez-Rodriguez, Song (2016). Recurrent Marked Temporal Point Processes. KDD. https://www.kdd.org/kdd2016/papers/files/rpp1081-duA.pdf
9. Mei & Eisner (2017). The Neural Hawkes Process. NeurIPS. https://arxiv.org/abs/1612.09328

10. Zuo, Jiang, Li, Zhao, Zha (2020). Transformer Hawkes Process. ICML (PMLR 119). http://proceedings.mlr.press/v119/zuo20a/zuo20a.pdf

11. Zhang, Lipani, Kirnap, Yilmaz (2020). Self-Attentive Hawkes Processes. ICML (PMLR 119). https://arxiv.org/abs/1907.07561

12. Shchur, Biloš, Günnemann (2020). Intensity-Free Learning of Temporal Point Processes. ICLR. https://arxiv.org/abs/1909.12127

13. Trivedi, Dai, Wang, Song (2017). Know-Evolve: Deep Temporal Reasoning for Dynamic Knowledge Graphs. ICML (PMLR 70). https://arxiv.org/abs/1705.05742

14. Trivedi, Farajtabar, Biswal, Zha (2019). DyRep: Learning Representations over Dynamic Graphs. ICLR. https://openreview.net/forum?id=HyePrhR5KX

15. Xue et al. (2024). EasyTPP: Towards Open Benchmarking Temporal Point Processes. ICLR. https://arxiv.org/abs/2307.08097

Survival analysis and ranking

16. Kaplan & Meier (1958). Nonparametric estimation from incomplete observations. JASA 53:457–481.

17. Cox (1972). Regression models and life-tables. JRSS-B 34:187–220.

18. Fine & Gray (1999). A proportional hazards model for the subdistribution of a competing risk. JASA 94:496–509.

19. Graf, Schmoor, Sauerbrei, Schumacher (1999). Assessment and comparison of prognostic classification schemes for survival data. Statistics in Medicine 18(17–18):2529–2545.

20. Antolini, Boracchi, Biganzoli (2005). A time-dependent discrimination index for survival data. Statistics in Medicine 24(24):3927–3944. https://onlinelibrary.wiley.com/doi/10.1002/sim.2427

21. Katzman, Shaham, Cloninger, Bates, Jiang, Kluger (2018). DeepSurv. BMC Medical Research Methodology 18:24. https://link.springer.com/article/10.1186/s12874-018-0482-1

22. Lee, Zame, Yoon, van der Schaar (2018). DeepHit. AAAI. https://cdn.aaai.org/ojs/11842/11842-13-15370-1-2-20201228.pdf

23. Lee, Yoon, van der Schaar (2019). Dynamic-DeepHit. IEEE Trans. Biomedical Engineering 67(1).

24. Ross, Das, Sciro, Raza (2021). CapitalVX: a machine learning model for startup selection and exit prediction. J. Finance and Data Science. https://www.sciencedirect.com/science/article/pii/S2405918821000040

Reference implementations. `pycox` and `scikit-survival` (DeepHit, Cox, Brier, concordance); `EasyTPP` (RMTPP / NHP / THP baselines).

Related documents

`paper/investment_case_study.pdf` (the model and proofs); `SURVIVAL_RANKING.md` (survival method in full); `NEXT_FUNDING_DESIGN.md` (the ranking application); `README.md` (the package).